Introduction

In modern software development, structured processes are required to ensure that system requirements are properly met and fully implemented into design and working code. For this assignment, we are tasked with following the main stages of software development: requirements engineering, software design, and implementation in order to make a simple patient record system or SPRS. The system must be able to register new patients, retrieve records, update information, delete records, and ensure unique and immutable IDs.

Requirements Traceability

Traceability plays an important role in this assignment. Requirements are mapped to specific methods within the *patient_registry* class with comments throughout the code indicating which requirements are being met. For example, the ability to register a new patient is implemented through *register_patient*, getting the patient data is implemented through *get_patient*, and so on. Additionally, *update_patient_name* allows for the patient names to be updated by using their unique patient ID but does not allow for their ID to be changed. By maintaining clear connections between requirements, design documentation, and implemented code we (working as developers) can verify that all requirements are met and properly implemented into our system. This also allows for easy fixes when a problem arises, as we are able to recognize the issue and jump to that part of the code. Traceability also allows for easy identification of which parts of a system are affected in case a requirement changes in the future. Traceability ensures completeness and easier maintenance of not only this assignment but also of any project or system that uses this approach.

Design Explanation

The requirements for this project were tight and enforced a secure design. Going into our design, we kept this in mind. We did this mainly by following the single responsibility principle. As we transitioned from written requirements into system design, this philosophy led to two main sub-systems: the main driver and the patient registry class. These two subsystems have one responsibility, either managing data or updating data. The patient registry class was responsible for managing the internal database. This internal database was stored with a dictionary. The

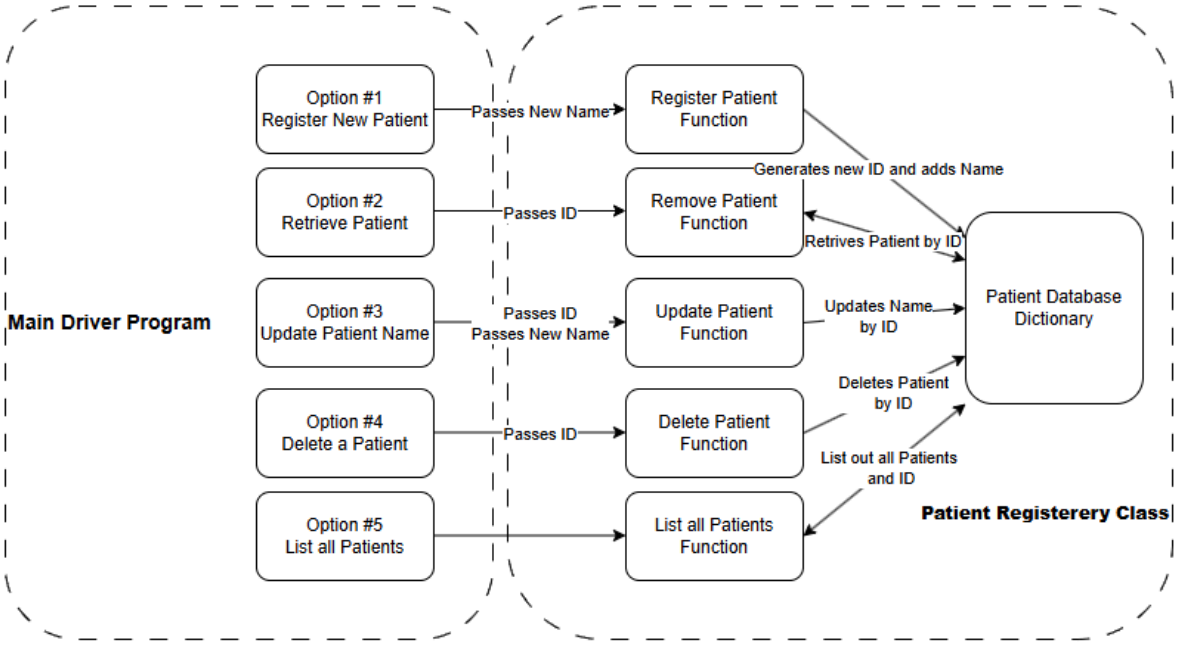
reason a dictionary was chosen for this project was to ensure unique pairs. While parallel arrays would be able to store ID alongside patent name, only a dictionary could be used to ensure that every patient is matched with one ID and vice versa. While our registry sub-system had access to manage the data, the main driver system had the ability to actually update the database in response to user input. To ensure safe data interaction from REQ-03, the main system presents the user with a set of options limiting the exposure the user has to the database. In these options, if a user goes down a path to retrieve or update a patient whose ID does not exist, the error is handled and displayed to the user in the main system. Because of this, IDs are completely immutable to the user.

Implementation Discussion

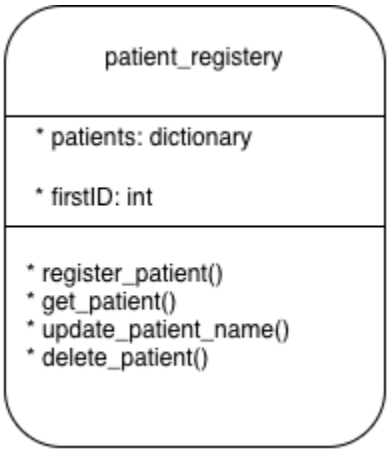
As stated before, we used a dictionary to represent our patient database. For the implementation, we defined a class called "patient_registry". This class has two attributes, a dictionary that holds the patient's names (values) and patient's IDs (keys). This decision was made because of the functionality of Python dictionaries. They are simple to use and work. Dictionaries allow for different data values to have unique keys, which work perfectly with REQ-01, and they are also very simple to use. All data input is done through the setter, which takes a patient's name and assigns a unique ID/key by incrementing by one from the last ID. Data validation is done through a single if statement, another reason why a dictionary is used. The code checks to see if the entered ID is valid and raises an error if not. Dictionaries also allow the retrieval of a value, just by entering the key. A user can easily change the value this way too. An assumption that we made was how to prevent modification of a patient ID. REQ-03 does not specify specifically how to avoid this, so we assumed that data integrity could be achieved by simply not allowing a user any access to change the dictionary. No method allows any change for a patient ID.

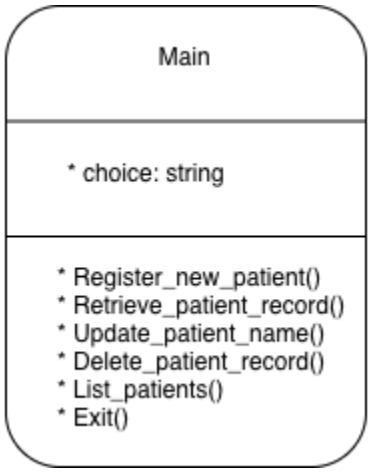
Diagrams

High-Level Architecture Diagram:



Class Diagram:





.

Screenshot evidence

Registering a Patient

```
Welcome to the Patient Registry System.

Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system


Enter your choice (1-6): 1
Register a new patient, type the patient's name, or type 'exit' to return to the main menu.
Enter patient's name: Tyler
Patient 100 registered successfully!
```

Retrieving a Patient (Success)

```
Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system


Enter your choice (1-6): 2
Retrieve a patient record by ID, type the patient's ID, or type 'exit' to return to the main menu.
Enter patient's ID: 100

-----------------------------------------------
Patient Record --------------------------------
Patient Name: Tyler
Patient ID: 100
-----------------------------------------------
```

Retrieving a Patient (Failure)

```
Welcome to the Patient Registry System.

Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system


Enter your choice (1-6): 2
Retrieve a patient record by ID, type the patient's ID, or type 'exit' to return to the main menu.
Enter patient's ID: 101
'ID# 101 was not found in the registry!'

Welcome to the Patient Registry System.

Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system


Enter your choice (1-6):
```

Updating Patient's Name

```
Welcome to the Patient Registry System.

Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system


Enter your choice (1-6): 3
Update a patient's name by ID, type the patient's ID, or type 'exit' to return to the main menu.
Enter patient's ID: 100
Type the patient's new name, or type 'exit' to return to the main menu.
Enter patient's new name: Karson
Patient 100 updated successfully!
New name: Karson
```

Updating Patient's Name (Failure)

```
Welcome to the Patient Registry System.

Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system


Enter your choice (1-6): 3
Update a patient's name by ID, type the patient's ID, or type 'exit' to return to the main menu.
Enter patient's ID: 101
Type the patient's new name, or type 'exit' to return to the main menu.
Enter patient's new name: Craig
'ID# 101 was not found in the registry!'

Welcome to the Patient Registry System.

Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system


Enter your choice (1-6):
```

Deleting Patient (Success)

```
Welcome to the Patient Registry System.

Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system


Enter your choice (1-6): 4
Delete a patient record by ID, type the patient's ID, or type 'exit' to return to the main menu.
Enter patient's ID: 100
Patient 100 deleted successfully!
```

Deleting Patient (Failure)

```
Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system


Enter your choice (1-6): 4
Delete a patient record by ID, type the patient's ID, or type 'exit' to return to the main menu.
Enter patient's ID: 100
'ID# 100 was not found in the registry!'

Welcome to the Patient Registry System.

Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system


Enter your choice (1-6): |
```

Listing All Patients

```
Welcome to the Patient Registry System.

Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system


Enter your choice (1-6): 5
List of all patient records:
Patient ID: 101, Patient Name: Tyler
Patient ID: 102, Patient Name: Craig
Patient ID: 103, Patient Name: Karson

Welcome to the Patient Registry System.

Please select an option:

    1. Register a new patient
    2. Retrieve a patient record by ID
    3. Update a patient's name by ID (ID cannot be changed)
    4. Delete a patient record by ID
    5. List all patient records
    6. Exit the system
```

Source Code

main.py

```
import patient_registry as pr

if __name__ == "__main__":

  registry = pr.patient_registry()

  while True:

      print("""
Welcome to the Patient Registry System.

Please select an option:

  1. Register a new patient
  2. Retrieve a patient record by ID
  3. Update a patient's name by ID (ID cannot be changed)
  4. Delete a patient record by ID
  5. List all patient records
  6. Exit the system

  """)

      choice = input("Enter your choice (1-6): ")

      ### REQ-03: No choice or setter available to update patient ID, thus
      ### the system shall ensure data integrity by preventing the modification of a
      ### Patient ID once it has been assigned.

      match(choice):

          case '1':
```

```
            print("Register a new patient, type the patient's name, or type 'exit' to return to the main menu.")
            name = input("Enter patient's name: ")

            if name.lower() == 'exit':
                continue

            new_id = registry.register_patient(name)

            print(f"Patient {new_id} registered successfully!")

        case '2':
            print("Retrieve a patient record by ID, type the patient's ID, or type 'exit' to return to the main menu.")
            id = input("Enter patient's ID: ")

            if id.lower() == 'exit':
                continue

            try:
                record = registry.get_patient(int(id))
                print(
f"""
----------------------------------------------
Patient Record --------------------------------
Patient Name: {record[int(id)]}
Patient ID: {id}
----------------------------------------------
""")
            except KeyError as e:
                print(e)

            continue

        case '3':
            print("Update a patient's name by ID, type the patient's ID, or type 'exit' to return to the main menu.")
            id = input("Enter patient's ID: ")
```

```
            if id.lower() == 'exit':
                continue

            print("Type the patient's new name, or type 'exit' to return to the main menu.")
            new_name = input("Enter patient's new name: ")

            if new_name.lower() == 'exit':
                continue

            try:
                registry.update_patient_name(int(id), new_name)
                print(f"Patient {id} updated successfully!")
                print(f"New name: {new_name}")
            except KeyError as e:
                print(e)

            continue

        case '4':

            print("Delete a patient record by ID, type the patient's ID, or type 'exit' to return to the main menu.")
            id = input("Enter patient's ID: ")

            if id.lower() == 'exit':
                continue

            try:
                registry.delete_patient(int(id))
                print(f"Patient {id} deleted successfully!")
            except KeyError as e:
                print(e)

            continue

        case '5':
```

```
            if len(registry.patients) == 0:
                print("No patient records found.")
                continue

            print("List of all patient records:")

            for id, name in registry.patients.items():
                print(f"Patient ID: {id}, Patient Name: {name}")

            continue

        case '6':
            print("Exiting the system. Goodbye!")
            break
```

patient_registry.py

```
class patient_registry:

  def __init__(self):
      self.patients = {}
      self.firstID = 100

  ### REQ-01: The system shall allow the creation of
  ### a new patient record with a name and a unique Patient ID.
  def register_patient(self, name):

      self.patients[self.firstID] = name
      self.firstID += 1

      return self.firstID - 1
```

```python
### REQ-02: The system shall retrieve a patient's
### record using their unique Patient ID.
def get_patient(self, patient_id):

    if patient_id in self.patients.keys():
        return {patient_id: self.patients[patient_id]}
    else:
        raise KeyError(f'ID# {patient_id} was not found in the registry!')

### REQ-04: The system shall allow updating a patient’s name using
### their Patient ID, while keeping the Patient ID unchanged.
def update_patient_name(self, patient_id, new_name):

    if patient_id in self.patients.keys():
        self.patients[patient_id] = new_name
    else:
        raise KeyError(f'ID# {patient_id} was not found in the registry!')

### REQ-05: The system shall allow deleting a
### patient record using their Patient ID. If the ID
### does not exist, the system shall display
### an appropriate error message.
def delete_patient(self, patient_id: str):



    if patient_id in self.patients.keys():
        self.patients.pop(patient_id)
        return True

        ### REQ-05: If the ID does not exist,
        ### the system shall display an appropriate error message.
    else:
        raise KeyError(f'ID# {patient_id} was not found in the registry!')
```

Github Link

https://github.com/Tyler-Kelly1/SEAssingment3

Team Member Contribution

Tyler - Github management, Screenshots, Design Explanation, high-level architecture diagram

Karson - Design Implementation, Class Diagram

Craig - Assignment Introduction, Requirements Traceability